

Feature: The worker's text

From build-plan: feature 8c

Branch: feature/08c-the-worker-s-text

Status: verified. Steps 8c.1 to 8c.3 built, tested and reviewed step by step (audit, independent review, re-reviews), 2026-10-07 to 2026-10-08. F-251 accepted by Frank 2026-10-08. No P0 or P1 open or fixed. The checkpoint for the final review.

Goal

Pedro paints for Summit. He has no Google calendar connected and no email of his own at the business, so today nothing tells him that Jane booked an estimate with him for Tuesday at 7:30. With this feature he gets a text from the business's own number: what landed on his day, when, who and where. And because a day changes, he also hears when that booking moves or comes off his day, so he never drives to an empty house.

This is the last resort feature 6 promised (decision 8: one email per business now; decision 11: the booked worker is told by text with feature 8). It reuses 8b's text door, settings and job rules; nothing new reaches Twilio.

Everything is the business's own setting (plan decision 30, `AGENTS.md` "The client decides everything"): which people get texts, on which phone, and for which of the three changes. Nothing is on by default; a person with no settings gets no texts.

In scope

- Each person's worker-text settings in their own table: their phone and three switches (added to their day, moved, taken off their day), set at client setup until Settings (feature 12).
- The text when a booking lands on a person's day: a new booking, from the form or made by the owner, and a move onto them from another person.
- The text when their booking moves to another time, staying theirs.
- The text when a booking comes off their day: cancelled, or moved to another person.
- Each one a job on the 8a runner, doing what is still true when it runs, retried like 8b's texts and never sent twice.
- Each worker text that went recorded on the contact's timeline (`sms_sent`).

Out of scope

- The Settings screen for these switches (feature 12); values are set at client setup. Step 8c.1 adds them to item 12's text settings line.
- A person's own work email and its booking notification (feature 12, feature 6 decision 8).
- Naming the worker when they reply to the business's number: their reply is passed on like any reply (8b decision 9), showing their number, not their name. Only a note for later, feature 12 can teach `findReplySenderName` people's phones.
- The customer's phone in the worker's text, and the worker messaging the customer: relayed worker messages are the idea parked for feature 19.
- Reminders to the worker before the appointment: only a note for later, one more switch beside these three if a client wants it.
- Delivery receipts and a screen of failed texts (feature 11 can show them).
- Buying numbers and putting keys on Railway: nothing new here, 8b's deploy notes cover it.

Decisions made in the spec

1. **Per person, nothing on by default** (plan decision 30). A `worker_text_settings` row per person holds their phone and three switches. No row, no texts. "When a business runs on one email" is the reason the text exists, not a condition the code checks: the business turns a person's texts on, and when feature 12 adds work emails it picks email, text or both for each person. Rejected: one switch for the whole business (Pedro without Google wants texts, Maria with Google wants none); texting only people without a Google connection, which fixes a rule for every business. 2. **The worker hears about every change to their day, not only a new booking** (the one place the critique widened the plan's line). The build plan says "a text with the booking", and feature 8 says the text "keeps them in the loop". A worker told about Tuesday 7:30 and never told it moved to Wednesday, or to Maria, or was cancelled, drives to an empty house: the customer's emails tell Jane and the business inbox, never him. So three texts, each its own switch: **added** (a new booking, or a move onto them from another person), **moved** (their booking, a new time), **taken off** (cancelled, or moved to another person). Rejected: the new-booking text only, which leaves the worker with a wrong day after any change. 3. **Any booking, the owner's too**. A booking the owner makes sends the business no notification (feature 6 decision 10: the owner already knows), but the worker did not make it, so their text goes the same as for a booking from the form. 4. **The text says what a worker needs to show up, in one piece, and never the customer's phone or link**. The business's name, what happened, the time in the business's zone, the customer's name, the service, the room when there is one, and the address:

- added: `Summit Painting: new booking Tue Oct 13, 7:30am. Jane Doe, Interior estimate, 1234 Long St NW Calgary`
- moved: `Summit Painting: moved to Wed Oct 14, 9:00am. Jane Doe, Interior estimate, 1234 Long St NW Calgary`
- taken off: `Summit Painting: off your day, Tue Oct 13, 7:30am. Jane Doe, Interior estimate` One billed piece, like 8b (decision 4: at most 160, the eight double-cost characters counted as two, plain ASCII through `plainText`). Too long, it is cut in this order: the business name loses

words from its end (`fitBusinessName`), then the address from its end, then the service; the time and the customer's name are never cut. The customer's phone is left out on purpose (Frank, 2026-10-03: a worker never sees the customer's number; messages go through the business). The booking page link is left out too: it is Jane's private link, and it cancels. The customer's name, service, room and address are typed by people, so they go through `plainText` and nothing else interprets them. 5. **Each text is a job that does what is still true when it runs** (8b decision 6). The job carries ids, the person, the booking's move number and, for taken off, the start that person had (a time, nothing personal). When it runs:

- added goes only if the booking is confirmed, not started, and still this person's; it says the booking's time now.
- moved goes only if the same, and no later move replaced it (that move sends its own).
- taken off goes only if this person is not on the booking now (cancelled or another person's), the start they had has not passed, and they knew of it: an added or moved text to them for this booking is on the timeline, or their added switch is off (they learn of bookings elsewhere, their Google or the owner). So nobody is told a booking left their day that they never heard was on it: Pedro to Maria and straight back before the jobs run tells Maria nothing, and a booking cancelled before its added text went sends no taken off either (found while building the Simulate page). A booking moved away and back to them sends no taken off; the move back sends added. Amended after 8c.3's review (Frank, 2026-10-07): "knew of it" means they believe it is on their day: their latest text about it was an added or moved one, newer than any taken off they got (F-243: no second "off your day" for a booking they already think is gone), or an added or moved text to them was tried and never known to have gone or failed (waiting for a retry, or given up after its last), so it may have reached them (F-245: a lost answer never leaves them driving to an empty house; read from the runner's job, which stays for both; a retry that finds the booking no longer theirs still asks Twilio and records the text if it went, since its job ends there, F-249). A try that never reached Twilio counts too: when unsure, they are told. Accepted as is: a taken off says the time they had when it came off, which a skipped moved text may never have told them; the customer's name still tells them which booking it is. Also accepted (Frank, 2026-10-08, F-251, about 1 in 100 million texts): a lost answer, then the booking's time moved and cancelled in the seconds before the retry; the retry looks for the words with the new time, finds none, and no taken off goes.
- Every kind needs: the person active, their row with that switch on, the business's text settings (its number) and its time zone. Anything missing is one log line, nothing sent. (A phone a text cannot reach cannot be stored: 8c.1's database check refuses it, so 8c.2 has no such case.)
- added and moved are both skipped when the person was already told after this change: an added or moved text to them for this booking described the booking at or past this change's move number (the same entries "knew of it" reads; each records the move number its text described, built in 8c.3 instead of comparing times, which would rest on the database's clock and the change's agreeing). So Marco to Pedro and straight back before the jobs run texts Marco "new booking" once, not twice, and a moved text never repeats an added text that ran late and already said the new time (F-242, Frank, 2026-10-07; this replaces the repeat this decision first tolerated). 6. **From the business's own texting number, and never to a texting number.** A business without `text_settings` sends no worker texts. A worker's phone that is any business's texting number is never texted (it would arrive as a customer's

reply in that business, the F-199 leak): checked when the job runs with `findTextingBusiness`, logged without the number. Feature 12 refuses it on save. 7. **Retried like 8b's texts, never sent twice** (8b decisions 7 and 8). Up to 10 tries; refusals no retry can change are logged. Before a retry sends, `findSentText` looks for the same words from the business's number to the worker's since the change was saved (its time is in the job), and records that one instead. 8. **Recorded on the contact's timeline.** Each worker text that went is an `sms_sent` entry with `kind` `worker_added`, `worker_moved` or `worker_removed`, the booking id, the person's id and Twilio's id; never the number or the words. Feature 11 can then show "Pedro was texted". Rejected: no record, which leaves the owner only Twilio's logs.

Build loop

Steps are built one at a time on `feature/08c-the-worker-s-text`. Each step's plan gets Frank's yes just before it is built. After that yes nothing stops until the review: build, tests, tick the box, the build log entry, commit with the step number and push to the feature branch, `/audit` scoped to the step, then the independent review (`workflow.stepReview: "every"`, `workflow.checkpointCommits: "enabled"`). Findings are talked through after the review; P0/P1 are fixed before the next step. `/complete` makes the merge commit, on Frank's yes.

No new package. Nothing touches Twilio's live account or Railway; every test fakes Twilio.

Build steps

- ☒ **8c.1 Who gets the worker's texts.** The `worker_text_settings` table and its migration (0021, generated from the schema): one row per person, their phone and the three switches, no defaults. Its validation schema in `packages/shared` (the phone made textable, each switch required). The seed gives one Summit Painting (dev) person a made-up 555 phone with all three on, and Riverbend Clinic (dev) no rows. Build-plan item 12's text settings line gains each person's phone and switches, refused on save when the phone is any business's texting number. **Done when:** `db:migrate` and `db:seed` run on a fresh `scheduleads_dev` and the seed's second run changes nothing; the shared tests pass, including: a phone in each shape 8b accepts is saved as `+1` and ten digits, a phone outside North America is refused, a missing switch is refused; the database refuses a phone not in the `+1` shape and a row pointing at another business's person; the backend tests still pass.
- ☒ **8c.2 The text when a booking lands on a worker's day.** The added text's wording (`renderWorkerText`, decision 4) and the one function that sends any worker text (`sendWorkerText`, decisions 5 to 8), the `worker_text` job, and `book-time.ts` adding it in the booking's own transaction, for the form and the owner alike. **Done when:** the shared and backend tests pass, including: a booking from the form texts its person from the business's number, and an owner's booking does too; the text fits one piece and is cut in decision 4's order, the time and the name never; a person with no row, the switch off, inactive, or a phone that is a texting number gets nothing and one log line; so does a business without text settings or a time zone; a booking cancelled, started, or

moved to another person before the job ran sends nothing; one moved to another time before it ran says the new time; a resent form adds no job; a retry that finds the text already went records it and sends nothing; a refusal no retry can change is not retried; each text that went is one `sms_sent` entry with `worker_added` and no number or words.

- ☒ **8c.3 Moved, and taken off the worker's day.** The two other wordings, and `move-booking.ts` and `cancel-booking.ts` adding the jobs in their own transactions: a move that keeps the person adds moved; a move to another person adds taken off for the first and added for the second; a cancel adds taken off. **Done when:** the backend tests pass, including: a move that keeps the person texts them the new time; a later move replaces an earlier moved text, which then sends nothing; a move to another person texts the first "off your day" with the time they had and the second "new booking"; a booking moved away and back sends the first no taken off, and one moved away and straight back before the jobs ran texts the first "new booking" once, not twice; a moved text that a late "new booking" already covered sends nothing; one moved to a second person and straight back before the jobs ran texts the second nothing; a cancel texts the person "off your day"; a booking cancelled before its added text went sends no taken off, unless the person's added switch is off; a second cancel press adds nothing; taken off for a start already passed sends nothing; each switch off stops only its own text; each text that went is one `sms_sent` entry of its kind.

Files / areas

- `packages/shared/db/text-tables/worker-text-settings-table.ts` , exported through `packages/shared/db/index.ts` .
- `packages/shared/migrations/0021_*.sql` , from `db:generate` .
- `packages/shared/zod-validation/text-validation-schemas/worker-text-settings-validation-schema.ts` and its test, beside `text-settings-validation-schema.ts` .
- `packages/shared/scripts/seed-dev.ts` .
- `backend/lib/text/` : `find-worker-text-context.ts` (the booking, its person, the customer's name, the service, the room, the address, the business's name and zone, read inside the business), `render-worker-text.ts` and its test, `send-worker-text.ts` . Reused unchanged: `sendText` , `findSentText` , `findTextSettings` , `findTextingBusiness` , `plainText` , `fitBusinessName` , `textPieceLength` , `formatTextTime` , `texttablePhoneNumber` , `logTextNotSent` .
- `backend/lib/jobs/` : `worker-text-job.ts` , `enqueue-worker-text.ts` , `job-names.ts` (`workerText`), `job-tasks.ts` , and the job's test.
- `backend/lib/booking/book-time.ts` (8c.2), `move-booking.ts` and `cancel-booking.ts` (8c.3), and their route tests.
- `blueprint/build-plan.md` , item 12's text settings line (8c.1).

Data / contracts

Table `worker_text_settings` (8c.1):

| Column | Type | Rule | |---|---|---| | `personId` | text, primary key | one row per person | | `organizationId` | text, not null | with `personId`, a foreign key to `resource (organizationId, id)`, cascade: only a person of the same business | | `phone` | text, not null | +1 and ten digits, area code and exchange starting 2 to 9 (the same check as `text_settings`) | | `addedOn` | boolean, not null | no default | | `movedOn` | boolean, not null | no default | | `removedOn` | boolean, not null | no default | | `updatedAt` | timestamptz, not null | now, and on every update |

A place never gets a row: the validation schema's callers (the seed now, Settings later) only offer people, and the job reads only a booking's `personId`. The phone is not unique: two people may share one.

Job `worker_text` payload, ids and times only:

```
type WorkerTextJobPayloadType = {
  organizationId: string;
  bookingId: string;
  personId: string;           // who is told
  sequence: number;           // the booking's move number when the job was added
  changedAt: string;          // ISO, when the change was saved: the retry check's "sin
} & (
  | { kind: "added"; startsAt: null }
  | { kind: "moved"; startsAt: null }
  | { kind: "removed"; startsAt: string } // ISO, the start this person had
);
```

No job key: each change adds its own jobs; a resent form and a second cancel press change nothing, so they add none. Unknown kinds throw, as in `booking-text-job.ts`. Each booking's worker texts run in one lane, `worker-text-` and the last two characters of its id, as its calendar jobs do (8a, decision 5): one at a time, in the order added, so decision 5's rules read every text an earlier change sent (added in 8c.3: the runner works five jobs at once). A failed try waits behind later jobs of its lane.

Timeline: `sms_sent`, payload `{ bookingId, kind, personId, sequence, twilioSid }`, `sequence` the least the text told them, as the booking's move number (added in 8c.3, for decision 5): the booking as it is for a text sent now; for one a retry finds already went, the change it was sent for, since it went at some earlier try and may describe a later move (F-250: a lower number at worst lets a later "new booking" or "moved" repeat), `kind` one of `worker_added`, `worker_moved`, `worker_removed`. No number, no words.

Log lines through `logTextNotSent(bookingId, kind, reason)`: the reason in plain words, never a phone number, a name or an address.

Testing

- Shared: `npm run test --workspace=@scheduleleads-app/shared` (the validation schema, 8c.1).
- Backend: `npm run test --workspace=backend`, local Postgres migrated and seeded. The job tests use 8b's fake Twilio and the pinned `jobClock`; the booking, move and cancel route tests make their own people and rows and remove them. Run the full backend suite several times per step (flaky jobs only showed over 7 to 10 runs in 8a and 8b).
- Each case the Simulate page lets Frank play becomes a test of the same name in the step that builds its rule.
- No frontend change, so no frontend test and no browser test.

Notes for the AI

- Follow `send-booking-text.ts` for the shape of `sendWorkerText`: read afresh, the reasons as plain words through `logTextNotSent`, `findSentText` only when `attempt > 1`, `SendTextError` without `retry` logged and not thrown, anything else thrown so the runner retries.
- The phone shown in logs and timeline is never the worker's number.
- Check `record-activity` for how `sms_sent` payloads are typed before adding the three worker kinds.
- `move-booking.ts` already knows `personChanged` and the row's old `personId` and `startsAt`; add the worker jobs beside `enqueueBookingTexts`.
- `cancel-booking.ts` adds its jobs only when the cancel changed something; the taken off job goes in that same branch.
- Feature 12 must refuse a worker phone that is any business's texting number on save (decision 6); 8c.1 writes that into build-plan item 12.

Implementation walkthrough

What was actually built, by area. The steps above are the plan; the build log records how each step went against it. This is the shape of the code the feature left behind, and the decisions in it that are not visible from the code alone.

shared: each person's settings (8c.1)

- `db/text-tables/worker-text-settings-table.ts` (migration 0021): one row per person, keyed by `personId`, with the phone and the three switches `addedOn`, `movedOn` and `removedOn`, none with a default. A composite foreign key on (`organizationId`, `personId`) to the resource table means a row can only belong to a person of the same business, and it goes with them. The phone is not unique: two people may share one.
- `db/text-tables/stored-textable-phone-pattern.ts`: the one stored shape of a texting phone (" +1" and ten digits, area code and exchange 2 to 9), now used by both `text_settings` checks and the new table's check, so the rule lives in one place (F-241).

- `zod-validation/text-validation-schemas/textable-number-validation-schema.ts` : a phone as people type it ("403 555 0148"), kept as Twilio texts it. 8b's text settings schema and the new worker schema both use it.
- The seed gives Pedro (painter) at Summit a made-up 555 phone with every switch on, parsed through the schema. Nobody else has a row, so the dev data also shows a person who gets no texts. Build-plan item 12 gained the line for these settings on the Settings screen.

backend: the jobs and their lane (8c.2, 8c.3)

- `lib/jobs/worker-text-job.ts` : the payload carries ids and times only: the booking, the person, the kind, the booking's move number (`sequence`) and `changedAt` when the change was saved; a "removed" text also carries the start the person had. A job of no known kind throws instead of being guessed into another text.
- `lib/jobs/worker-text-lane-of.ts` : every worker text of one booking runs in one of 256 lanes, named by the last two characters of its id. Two texts of a booking never run at once and run in the order they were added, which is what lets "already told" and "knew of it" read the timeline safely.
- `lib/jobs/enqueue-worker-text.ts` : added inside the change's own transaction (8a, decision 1), always in that lane, whatever the person's settings say now; the job reads them when it runs.
- `lib/booking/book-time.ts` adds an "added" job for every booking, from the form or the owner alike (decision 3). `move-booking.ts` adds "moved" for the same person, or "removed" for the old person plus "added" for the new one. `cancel-booking.ts` adds "removed" only from a cancel that changed something, so a second press adds nothing.

backend: the one sender (8c.2, 8c.3)

- `lib/text/send-worker-text.ts` is the only place a worker text is decided and sent. In order: the booking afresh (`find-worker-text-context.ts` , inside its own business); whether the booking still calls for this text (`whyNotDue` : confirmed, still theirs, not started, not replaced by a later move; for "removed", not theirs again and the time they had not passed); the person's row, active and that switch on; what they were already told (`whyNotNews`); the business's text settings and time zone; the phone not being any business's texting number (`findTextingBusiness` , decision 6).
- What a person was told is read from the customer's timeline by `find-worker-texts-told.ts` : the move numbers their "new booking" and "moved" texts described, and those their "off your day" texts were sent at. "New booking" and "moved" are skipped once a text told them of this change or a later one (F-242), compared by move number rather than by time, so nothing rests on two clocks agreeing.
- "Off your day" goes only to someone who believes the booking is on their day: their latest text about it was "new booking" or "moved", newer than any "off your day" (F-243); or their "new booking" switch is off, so they learn of bookings elsewhere; or a "new booking" or "moved" text was tried and never known to have gone or failed. That last one is read from the runner's own table by `lib/jobs/has-worker-text-in-doubt.ts` , since the job is the only record of a try whose answer was lost (F-245, F-248).

- A retry that finds the booking no longer the person's still asks Twilio whether an earlier try went, and records it, because its job ends there and the "off your day" after it would otherwise lose the only sign (F-249). Every retry checks with `findSentText` since `changedAt` before sending. The recorded `sequence` is the least the text told them: the booking as it is for a text sent now, the change it was sent for for one a retry finds (F-247, F-250).
- `lib/text/render-worker-text.ts` writes the three texts in one billed piece. Too long, the business name loses words first (`fitBusinessName`), then the address from its end, then the service; the time and the customer's name are never cut. "Off your day" carries no room or address. `join-cut-words.ts` was pulled out of `fit-business-name.ts` so both cut the same way and never end on a joining mark.
- Each text that went is an `sms_sent` timeline entry with kind `worker_added` , `worker_moved` or `worker_removed` , the booking, the person, the move number and Twilio's id; never the phone or the words.

tests

- `backend/lib/jobs/worker-text-job.test.ts` (975 lines) drives the real path against the seeded dev database with Twilio faked: each switch, each kind, away and back, quick double moves, cancel before and after "added", lost answers and retries in every order traced in review. Retries are held and released (`holdRetries` / `releaseRetries`) because the runner breaks no tie between jobs due at the same moment.
- `render-worker-text.test.ts` covers the cut order and the one-piece limit; `worker-text-settings-rules.test.ts` the database check and the foreign key (another business's person, a malformed phone, a shared phone in one business); the shared schema test covers typed phones.

Non-obvious decisions taken while building

- The move number replaced a time comparison for "already told" in 8c.3, so a late "new booking" and a "moved" never repeat each other.
- Known limits, accepted by Frank: F-251 (a lost answer, then a time move and a cancel before the retry, about 1 in 100 million texts, gets no "off your day"). F-252, found by the final review and left open for Frank: a doubtful try keeps counting after an "off your day" went, so a booking that comes back and leaves again before its texts run can send a second one, never a missed one.

Findings

8c/F-240 [P3] closed - The shared-phone test puts the two people in different businesses, so a one-phone-per-business rule would still pass it

File: `backend/lib/text/worker-text-settings-rules.test.ts:86-91` **Found:** 2026-10-07 by independent step review (scope: 8c.1, ddca0e1..e165362; lenses: quality, security, performance, tests) **Why it matters:**

The contract says "The phone is not unique: two people may share one", and the table comment says the same. The test that pins it makes Pedro in business "share-first" and Pedro in business "share-second", so it only proves there is no unique index on `phone` alone. A `unique(organizationId, phone)` added later (the likely mistake, since 8b's `text_settings` has unique numbers) would pass it, while the real case is inside one business: two of Summit's painters on the crew lead's phone. The test name promises more than it checks. **Suggested fix:** Give the second person to the same business (a second `resource` row in `first.organizationId`) and insert both rows with the same phone. Keep the cross-business case as a second line if wanted. **Resolution:** Fixed 2026-10-07 with Frank's yes: the test is now "two people of one business may share one phone" (Carlos added to Pedro's business, both rows the same phone). Proved: a temporary unique ("organizationId", phone) on the dev database failed it, and it passed again once dropped. The cross-business line was not kept: nothing promises it beyond the phone not being unique on its own, which the one-business case already covers. Re-review of 8c.1's fixes (2026-10-07): closed. `worker-text-settings-rules.test.ts:86-94` now makes Carlos a second `resource` in Pedro's own business (`pedro.organizationId`) and inserts both rows with the same phone, so a `unique(organizationId, phone)` would refuse the second insert; Carlos goes with the tagged business in `afterAll` (the organization delete cascades through resource). The dev database keeps no leftover from the temporary-unique proof: `worker_text_settings` has only its pkey, `person_fk`, `phone_check` and not-nulls. Backend tests 747/747 passed three runs in a row. Nothing new introduced.

8c/F-241 [P3] closed - NORTH_AMERICAN_NUMBER is a regex pattern, not a number, and the same rule now lives in two shapes

File: `packages/shared/db/text-tables/north-american-number.ts:4`; `packages/shared/helpers/textable-phone-number.ts:6` **Found:** 2026-10-07 by independent step review (scope: 8c.1, ddca0e1..e165362; lenses: quality, security, performance, tests) **Why it matters:** 8c.1 lifted the constant out of `text-settings-table.ts` into its own file so a second table can share it, which makes it a named, imported thing. The import reads as "a North American number", but it is the source of a Postgres regular expression for a stored phone ("+1" and ten digits), and the file sits among the tables. The coding standards ask that the imported name carry the full meaning. The rule it encodes also exists as `NORTH_AMERICAN` in `textable-phone-number.ts` (the typed shape, before "+1"), so the two are kept in step only by a comment. **Suggested fix:** Rename to say what it is, for example `STORED_TEXTABLE_PHONE_PATTERN` in `stored-textable-phone-pattern.ts`, and keep the pointer comment to `textable-phone-number.ts`. A rename only: the SQL in the migrations is unchanged, so no new migration. **Resolution:** Fixed 2026-10-07 with Frank's yes: renamed to `STORED_TEXTABLE_PHONE_PATTERN` in `packages/shared/db/text-tables/stored-textable-phone-pattern.ts`, keeping the pointer to `textable-phone-number.ts`; both tables import it. A rename only: `db:generate` reports "No schema changes", backend build clean, shared 154 and backend 747 tests passed (3 runs). Re-review of 8c.1's fixes (2026-10-07): closed. The diff is a pure rename (value unchanged) that keeps the pointer comment; `git grep` finds no `NORTH_AMERICAN_NUMBER` or `north-american-number` outside this ledger entry, and both `text-settings-table.ts` and `worker-text-settings-table.ts` import the new name. `db:generate` reports "No schema changes, nothing to

migrate" and wrote no file; backend build, `format:check`, shared 154/154 and backend 747/747 (three runs) pass. Only a stale, gitignored `packages/shared/dist/db/text-tables/north-american-number.*` from an earlier build remains on this machine; `./db` does not export it and nothing imports it. Nothing new introduced.

8c/F-242 [P3] closed - The added rule cannot tell a superseded added job, so a booking moved away and straight back will text its person the same "new booking" twice once 8c.3 adds jobs on moves

File: backend/lib/text/send-worker-text.ts:40-44; backend/lib/jobs/worker-text-job.ts:20-22 **Found:** 2026-10-07 by independent step review (scope: 8c.2, b3b655c..7b24848; lenses: quality, security, performance, tests) **Why it matters:** Not a fault in 8c.2 as built (today only `book-time.ts` adds a worker job, so each booking has one added job). It is a trap for 8c.3. The added job sends whenever the booking is confirmed, not started and still this person's (decision 5 as written); the payload's `sequence` is written but dropped by `worker-text-job.ts`, and nothing looks at the person's earlier `worker_added` entries. 8c.3 adds an added job on a move onto a person, and its own Done when names the case "a booking moved away and back". Marco -> Pedro -> Marco before the jobs run leaves two added jobs for Marco (the booking's, sequence 0, and the move back's, sequence 2). Both pass every check at run time, both render the same words, and both are first tries, so `findSentText` never runs: Marco gets "Summit Painting: new booking Mon Oct 5, 9:00am. Jane Doe, ..." twice, against "never sent twice" in the spec's scope. (Decision 5 tolerates an added and a moved text repeating one time, which reads differently; two identical texts are not that case.) **Suggested fix:** Decide it in 8c.3's plan, before `move-booking.ts` adds its jobs: for example, added skips when a `worker_added` or `worker_moved` entry for this person and booking was recorded at or after this job's `changedAt` (the same entries decision 5's "knew of it" reads), with a test of the same name as the Simulate case. Or state in decision 5 that the duplicate is tolerated, so it is a choice and not a surprise. **Resolution:** Carried to 8c.3 with Frank's yes (2026-10-07): spec decision 5 now skips an added or moved text when an added or moved text to that person for that booking was recorded at or after the job's `changedAt`, and 8c.3's Done when has the two cases (moved away and straight back texts "new booking" once; a moved text a late "new booking" already covered sends nothing). Stays open until 8c.3 builds and tests it. Fixed 2026-10-07 in 8c.3: send-worker-text.ts skips an added or moved text when findWorkerNewsTold finds an added or moved entry for that person and booking at or past the job's move number (each sms_sent worker entry now records the move number its text described; a time comparison would rest on two clocks agreeing), and each booking's worker texts run in one lane (worker-text-lane-of.ts) so the rule reads every earlier text. Tests: "a booking moved away and back sends the first no taken off, and one moved away and straight back before the jobs ran texts the first \"new booking\" once, not twice" and "a moved text that a late \"new booking\" already covered sends nothing"; proved: removing the rule fails both. Audit of 8c.3 (2026-10-07): closed. Reviewed send-worker-text.ts, find-worker-news-told.ts, worker-text-job.ts, enqueue-worker-text.ts, worker-text-lane-of.ts, move-booking.ts and cancel-booking.ts at 44d36f0. Traced Marco -> Pedro -> Marco before the jobs run: the booking's added job (sequence 0) sends and records `sequence` 2 (the booking as the text described it), Pedro's added skips as another person's, Pedro's taken off skips as never told, and the move back's added

(sequence 2) finds $2 \geq 2$ and skips, so one "new booking". The rule suppresses only when an added or moved text already described the booking at or past the job's move number, and every later change adds its own job at a higher number (a cancel cannot be undone), so no due text is wrongly skipped. Every enqueue (book-time, move, cancel) goes through enqueueWorkerText, so all of a booking's worker texts share one lane. Both named tests pass; backend 785/785 three runs. Nothing new from the repair's logic; the finding number it put in a code comment is recorded separately as F-244.

8c/F-243 [P3] closed - "Off your day" counts any earlier added or moved text as "knew of it", so a person whose last text already said it left, or who never heard the time that is going, is told it is off at a time they never had

File: backend/lib/text/send-worker-text.ts:71-73; backend/lib/text/find-worker-news-told.ts:30 **Found:** 2026-10-07 by /audit (scope: 8c.3, c630b85..44d36f0; lenses: quality, security, performance, tests) **Why it matters:** Decision 5's aim is that nobody is told a booking left their day that they never heard was on it. The taken-off rule reads only `worker_added` and `worker_moved` entries and passes when any exists (`told.length > 0`), whatever came after. Two paths, on the same "before the jobs run" premise as the spec's own Simulate cases: (1) Pedro booked at 9:00 and told; moved to Maria and Pedro's "off your day, 9:00" goes; moved back to Pedro and cancelled before those jobs run. The move back's added skips (the booking is cancelled), then the cancel's taken off finds Pedro's sequence-0 entry and sends "off your day" at the new time: a second "off your day" for a booking he was last told had left, at a time he never heard. (2) Pedro told 9:00; moved to 10:00, then to Maria before the jobs run. The moved text skips (another person's), and the taken off sends "off your day, 10:00", a time he was never told. No one drives to an empty house (the text says it is off), so P3: a confusing text, not a missed one. **Suggested fix:** Decide it in the spec first. The smallest repair uses the move numbers 8c.3 already records: read `worker_removed` entries too, and let taken off go only when this person's newest added or moved entry is newer than their newest removed one; optionally also only when that entry described the time being taken off. Or state in decision 5 that these cases are tolerated, so it is a choice and not a surprise. **Resolution:** Independent review of 8c.3 (2026-10-07): agree, P3. Traced both paths at 44d36f0: whyNotNews (send-worker-text.ts:71-73) passes "off your day" on `told.length > 0`, and findWorkerNewsTold (find-worker-news-told.ts:30) never reads `worker_removed`, so path (1) sends a second "off your day" and path (2) names a time that was never texted. The same root cause, the ledger not modelling what the person last heard, also makes the told case of "moved away and back" send an identical "new booking" twice (the 8c.3 test at worker-text-job.test.ts:618-630 asserts exactly that, same words, same 9:00am); decision 5 chooses that ("the move back sends added"), so it is not filed separately, but one ordered read of added, moved and removed entries would settle both. Fixed 2026-10-07 with Frank's yes: "off your day" goes only when the person's latest text about the booking was an added or moved one, newer than any "off your day" they got (findWorkerTextsTold now reads `worker_removed` entries too), with its own log reason "the person was already told it is off their day". The second path (a time a skipped moved text never told them) is accepted as is and written into decision 5: the customer's name still says which booking. Test: "a person already told a booking is off their day gets no second off your day"; proved: the old rule fails it. Re-review of 8c.3's fixes (2026-10-07): left fixed. The defect is gone: send-worker-text.ts:102-105 lets "off your

day" go only when an added or moved entry's move number is above the newest worker_removed one, and putting back `told.onTheirDay.length > 0` fails "a person already told a booking is off their day gets no second off your day" (1 failed, 29 passed; file restored, same sha256). But the new comparison trusts the move number an "off your day" entry records, which on a retry recovered through `findSentText` is the booking's number when the retry ran, not when the text went, so it can now hide a later "off your day" that the old rule sent: F-247 path (1). Close together with F-247. Re-review of 8c.3's second fixes (2026-10-07): closed. The regression that held it open is gone: `send-worker-text.ts:136` records a text a retry finds with the job's move number (`text.sequence`), so F-247 path (1) records the lost "off your day" at 1, Marco's move-back "new booking" at 2 stays newer, and move 3's "off your day" goes (`worker-text-job.test.ts:892`, "a lost off your day found on its retry does not stop a later one after a new booking"). Sent-now entries record the booking's move number at send time, which only grows, and an on-day and an off-day text can never both go at the same move number (one needs the person on a confirmed booking, the other not), so the "latest text" comparison at `send-worker-text.ts:102-105` orders sent-now texts as they went. Backend 789/789, three runs. Nothing new from this rule.

8c/F-244 [P3] closed - The sender's file comment grew into a 15-line block restating the rules below it, and names a finding number

File: `backend/lib/text/send-worker-text.ts:1-15` (finding number at line 6) **Found:** 2026-10-07 by /audit (scope: 8c.3, c630b85..44d36f0; lenses: quality, security, performance, tests) **Why it matters:** `coding-standards.md` (Comments) asks for a short file-level comment, the reasoning beside the line it explains, and "No history in code comments (step numbers, finding numbers ...)". 8c.3 grew the header from 7 to 15 lines; lines 3-9 restate, rule by rule, what `whyNotDue` (48-63) and `whyNotNews` (65-77) decide, far from those lines, and line 6 ends with "F-242". **Suggested fix:** Keep the header to why the module exists (every worker text goes through it so the rules cannot drift; never to a texting number; the retry check; the timeline entry). Move the "new booking"/"moved" and "off your day" rules into a line or two above `whyNotDue` and `whyNotNews`, and drop "F-242" (the build log carries it). **Resolution:** Independent review of 8c.3 (2026-10-07): agree, P3. `send-worker-text.ts:1-15` restates the rules of `whyNotDue` (48-63) and `whyNotNews` (65-77) line by line far from them, which `coding-standards.md` "Comments" names as unwanted ("a long block at the top that explains lines far below it"), and line 6 carries "F-242", the only finding number in `backend/lib` (`grep -rn "F-[0-9]{3}" backend/lib`). Fixed 2026-10-07 (standards, no decision needed): `send-worker-text.ts`'s top comment is back to five lines naming the decisions, the rules sit beside `whyNotDue`, `whyNotNews` and the knew-of-it line, and no finding number is left anywhere in `backend/lib` code. Re-review of 8c.3's fixes (2026-10-07): closed. `send-worker-text.ts:1-5` is a five-line header naming decisions 5-8 only; the rules sit beside `whyNotDue` (39), `whyNotNews` (56-58) and the knew-of-it line (100-101); `grep -rnE "F-[0-9]{3}" backend/lib` (non-test, excluding dist) finds nothing. The new files' headers (`has-worker-text-in-doubt.ts`, `find-worker-texts-told.ts`) are four lines each and carry no history. Nothing new introduced.

8c/F-245 [P2] closed - A taken-off text runs ahead of a retrying "new booking" that may already have reached the person, finds no timeline entry, and is dropped for good

File: backend/lib/text/send-worker-text.ts:71-73, 90-91, 133-139; backend/lib/jobs/worker-text-lane-of.ts:1-4 **Found:** 2026-10-07 by independent step review (scope: 8c.3, c630b85..44d36f0; lenses: quality, security, performance, tests) **Why it matters:** "Knew of it" reads only recorded `sms_sent` entries, but decision 7 exists because a text can reach the person without being recorded: `sendText` throws a retryable `SendTextError` on a timeout or lost answer (`send-text.ts:55-70`) after Twilio may have taken it. The lane does not hold the order across a failure (`enqueue-job.ts:15` and the spec's Data / contracts: "A failed try waits behind later jobs of its lane"). Path: Pedro's "new booking" goes out at Twilio, the answer is lost, the job is rescheduled; if its retry's `findSentText` also fails (Twilio degraded), the waits grow toward hours. Jane cancels meanwhile. The cancel's taken-off job runs first in the lane, `told` is empty and Pedro's added switch is on, so it logs "the person never knew of this booking" and completes. The "new booking" retry then stops at `whyNotDue` ("the booking was cancelled", line 90-91) before the `findSentText` check at 133-139, so nothing records that he was told. Pedro has "new booking Tue 7:30" and never "off your day": the empty-house case the Goal names. `worker-text-lane-of.ts:2-4` claims the lane makes the rules "read every text an earlier change sent", which is untrue while an earlier job is retrying. No test covers a retrying added text followed by a cancel. **Suggested fix:** Decide in the spec, then: when a taken-off text finds `told` empty, look for an unfinished `worker_text` job for the same booking and person added before it (the runner's jobs table, as the lane test already queries) and, if one is still waiting, throw so the taken-off retries behind it; or, on attempt > 1 of an added or moved text that is no longer due, still run `findSentText` and record the text when it went, so a taken-off job retried after it sees the entry. Add a test of the same name: a "new booking" whose first try times out after Twilio took it, then a cancel, texts the person "off your day". **Resolution:** Fixed 2026-10-07 with Frank's yes: "knew of it" also counts an added or moved text to that person for that booking that was tried and still waits for a retry (`backend/lib/jobs/has-worker-text-in-doubt.ts`, read from the runner's own table), so it may have reached them and they get "off your day"; the lane comment now says a waiting retry is read from the runner. Test: "an off your day still goes when the new booking may have reached them and waits to retry"; proved: removing the check fails it. Re-review of 8c.3's fixes (2026-10-07): left fixed. The defect is gone: replacing `(await hasWorkerTextInDoubt(...))` with `false` fails that test (1 failed, 29 passed; file restored, same sha256). The private-table read is judged acceptable: graphile-worker's public `jobs` view (`sql/000017.sql`) has no payload, so `_private_jobs` is the only place the booking and person can be matched; the query is bound parameters plus `sql.identifier(jobSchema) ; ^0.18.0` admits 0.18.x only and the test above exercises the query, so a schema change on upgrade fails a test. `attempts > 0` does mean "started at least once" in 0.18 (`getJobs.js:186` increments on pick, `returnJobs.js:19` gives it back for a job fetched but never started); the "removed" job running now is excluded by kind, and lane serialization stops another of the booking's worker texts running beside it. But the repair let an "off your day" go on a waiting try, which with the recovered retry's move number can leave the person last told "off your day" for a booking that is theirs again (F-247 path (2), probed), and the check counts more than tries that may have reached them (F-248). Close together with F-247. Re-review of 8c.3's second fixes (2026-10-07): left fixed. The order this finding names (the taken-off runs while the retry waits) is handled and tested (`worker-text-job.test.ts:818`), and F-247 path (2) now ends on "new booking" (`worker-text-job.test.ts:913`). But the mechanism its Why names, a not-due retry stopping at `whyNotDue` (`send-worker-text.ts:86-87`) before `findSentText` (133-139) so nothing records it, still loses the text in the other

order: a retry due before the cancel but picked after it runs first, completes, leaves no job for `hasWorkerTextInDoubt`, and the cancel's "off your day" logs "the person never knew of this booking". Probed (temporary test, restored, same sha256): Marco gets only "new booking", no entries, no jobs left. Decision 5's amendment says of F-245 "a lost answer never leaves them driving to an empty house", which does not hold yet: filed as F-249. Close together with F-249. Re-review of 8c.3's third fixes (2026-10-08): closed. Both orders the defect named now end with "off your day": the cancel's job running while the retry waits reads the job (`send-worker-text.ts:164`, `has-worker-text-in-doubt.ts:13-25`), and the retry running first records the text it finds before its job ends (`send-worker-text.ts:123-143`, F-249, closed). Proved at HEAD: replacing line 164's `(await hasWorkerTextInDoubt(bookingId, text.personId))` with `false` fails "an off your day still goes when the new booking may have reached them and waits to retry" (`worker-text-job.test.ts:818`) and one other (2 failed, 32 passed; file restored, same sha256). Every defect the repair brought is settled: F-247, F-248, F-249 and F-250 closed, F-251 accepted by Frank in decision 5. The lane comment (`worker-text-lane-of.ts:3-5`) now says a waiting retry is read from the runner, which holds.

8c/F-246 [P3] closed - "Each switch off stops only its own text" never turns the taken-off switch off: ignoring `removedOn` passes every test

File: `backend/lib/text/send-worker-text.ts:42-46`; `backend/lib/jobs/worker-text-job.test.ts:738-753`

Found: 2026-10-07 by independent step review (scope: 8c.3, c630b85..44d36f0; lenses: quality, security, performance, tests) **Why it matters:** 8c.3's Done when lists "each switch off stops only its own text". The tests turn `addedOn` off (8c.2, and the cancel-elsewhere case) and `movedOn` off ("each switch off stops only its own text"), but no test sets `removedOn` to false. Proved: changing `removed:` `"removedOn"` to `removed: "active"` in `SWITCH_OF` leaves all 137 tests under `lib/jobs/worker-text-job.test.ts` and `lib/text` passing, so a business that turns a person's "off your day" texts off could keep sending them and nothing would catch it. The code is correct today; the Done when claim is unproved for one of its three switches. **Suggested fix:** Extend "each switch off stops only its own text" with a `removedOn`-off case (a cancel logs "the person has that text off" for `worker_removed` and sends nothing, while the added and moved texts still go). **Resolution:** Fixed 2026-10-07 (makes the agreed Done when hold): "each switch off stops only its own text" now also turns off only the "off your day" switch and checks the new booking and moved texts still go and the cancel's does not; proved: mapping that switch to the wrong field fails it. Re-review of 8c.3's fixes (2026-10-07): closed. `worker-text-job.test.ts:752-766` books with only Marco's `removedOn` off, moves and cancels, and checks the new booking and moved texts go, the cancel's does not, and "worker_removed not sent, the person has that text off" is logged. Proved again: `removed: "active"` in `SWITCH_OF` fails "each switch off stops only its own text" (1 failed, 138 passed over `worker-text-job.test.ts` and `lib/text`; file restored, same sha256). Nothing new introduced.

8c/F-247 [P2] closed - A retry recovered through `findSentText` records the booking's move number when it ran, not when its text went, and the fix's new "latest text" and "in doubt" rules now order texts by that number

File: backend/lib/text/send-worker-text.ts:102-107, 132, 137-141; backend/lib/text/find-worker-texts-told.ts:47-50 **Found:** 2026-10-07 by re-review of 8c.3's fixes (scope: 286078b..5e108e7; lenses: quality, security, performance, tests) **Why it matters:** `record` stamps `sequence: context.sequence`, the booking now. On a retry whose first try reached Twilio but lost its answer, `findSentText` finds the old text and records it at today's move number, though the person read it before any later change. Before the fix, "off your day" entries were never read and any added/moved entry passed, so this did not matter; the fix makes both rules depend on it. Probed with two temporary tests in `worker-text-job.test.ts` (restored, same sha256), each giving the lost try's retry a run time before the next change's job, as a runner restart or the up-to-two-second poll gap does: (1) Marco told at move 0; moved to Pedro (move 1), Marco's "off your day" taken but answer lost; moved back (move 2), "new booking" goes ahead of the retry; moved to Pedro again (move 3). The retry finds the old "off your day" and records it at 3; move 3's "off your day" then logs "the person was already told it is off their day". Marco's texts: new booking, off your day, new booking; entries `[added 0, added 2, removed 3]`; the booking is Pedro's and Marco's last text says it is his. (2) Marco's "new booking" taken, answer lost; moved to Pedro (move 1), the in-doubt check sends "off your day" (entry 1); moved back (move 2). The "new booking" retry runs first, finds its old text and records it at 2; move 2's "new booking" then logs "the person was already told after this change". Marco's texts: new booking, off your day; entries `[removed 1, added 2]`; the booking is his and his last text says it is off, so he misses it. At 44d36f0 both ended with the right last text. Same trigger class as F-245 (a lost answer), plus a narrow timing window, hence P2. **Suggested fix:** When a retry records a text `findSentText` found, record the move number of the change it was sent for (`text.sequence`) rather than the booking's now, so the timeline says what the text actually told them; with that, path (1) records 1 and move 3's "off your day" goes, path (2) records 0 and move 2's "new booking" goes. Add both paths as tests (each needs the lost try's run time moved ahead of the next change's job). **Resolution:** Fixed 2026-10-07 (makes decision 5 hold, no new decision): a text a retry finds already went is recorded with the change it was sent for (the job's move number), not the booking's move number when the retry ran; a text sent now still records the booking as it is. Tests: "a lost off your day found on its retry does not stop a later one after a new booking" and "a lost new booking found on its retry does not stop a later one after an off your day" (each asserts the retry asked Twilio once, and the person's last text); proved: recording the retry's current move number fails both, 3 runs of 3. The tests release the held retry ahead of the other due jobs, since the runner breaks no tie between jobs due at the same moment. Re-review of 8c.3's second fixes (2026-10-07): left fixed. The defect is gone: both probed paths are now tests that walk the order they name (`holdRetries` keeps the lost try's retry back, `releaseRetries` puts its `run_at` an hour before the next change's job, and each asserts one GET, the retry's `findSentText`), and changing `send-worker-text.ts:136` back to `record(sent, context.sequence)` fails both (2 failed, 30 passed; file restored, same sha256). `text.sequence` is a lower bound of what a found text told them, which can at worst let a later "new booking" or "moved" repeat; traced against every lane order I could build, it gives a wrong last text only through F-249's drop. But the repair's own comment (`send-worker-text.ts:124-125`) and the Data / contracts line it added say the found text "went at its first try" and so told "only the change it was sent for", neither of which is always true: F-250. Close together with F-250. Re-review of 8c.3's third fixes (2026-10-08): closed. A text a retry finds is recorded with the change it was sent for, on both the due path and F-249's not-due path, through the one helper (`send-worker-text.ts:116-121`, line 119 `record(sent,`

`text.sequence`). Proved at HEAD: changing line 119 to `record(sent, context.sequence)` fails both probed paths, "a lost off your day found on its retry does not stop a later one after a new booking" and "a lost new booking found on its retry does not stop a later one after an off your day" (worker-text-job.test.ts:892 and 913; 2 failed, 32 passed; file restored, same sha256). The only defect the repair brought, its wording, is F-250, closed; nothing new.

8c/F-248 [P3] closed - The in-doubt check counts every try of a "new booking" or "moved", including ones that never reached Twilio and jobs that gave up, though the spec and comment say "tried and still waits for a retry, so it may have reached them"

File: backend/lib/jobs/has-worker-text-in-doubt.ts:1-4, 21; blueprint/context/current-feature.md (decision 5, amendment) **Found:** 2026-10-07 by re-review of 8c.3's fixes (scope: 286078b..5e108e7; lenses: quality, security, performance, tests) **Why it matters:** The query is `attempts > 0` with no other condition. In graphile-worker 0.18 a job that used its last attempt stays in `_private_jobs` with `attempts = max_attempts` (only `is_available` turns false, sql/000011.sql:67), so a "new booking" that gave up counts as in doubt for that booking and person forever, not "still waits for a retry". And a try counts whatever failed it: send-text.ts retries refusals about the agency's own account and missing keys in production (its NEVER_RETRY comment), and any throw before sendText (a database error) also leaves `attempts = 1`; none of those can have reached the person. Path: Twilio refuses the account for an afternoon; Pedro's "new booking" keeps failing; Jane cancels; once the account is fixed, if the cancel's job runs before the added retry it finds the added job waiting, so Pedro gets a lone "off your day" for a booking he never heard of, the outcome decision 5's "knew of it" exists to prevent. The bias is the safe direction (an extra confusing text rather than a missed one), so P3, but the code does more than the amendment says. **Suggested fix:** Decide in decision 5: either accept it and say so ("any try, waiting or given up, counts"), rewording the file comment to match; or narrow the query to `attempts < max_attempts` for "still waits", and, if wanted, to tries whose `last_error` is a timeout, no answer or unreadable answer, the only failures after which Twilio may have taken the text. **Resolution:** Fixed 2026-10-07 (keeps Frank's rule of thumb for F-245, "when unsure, tell him"): accepted and reworded rather than narrowed. A job that gave up after its last try may also have reached them, so it still counts; decision 5 and has-worker-text-in-doubt.ts now say "tried and never known to have gone or failed, waiting for a retry or given up", and that a try which never reached Twilio counts too, in the safe direction. Re-review of 8c.3's second fixes (2026-10-07): closed. The code and its words now agree: has-worker-text-in-doubt.ts:1-5 says a try that is waiting or gave up counts, "when unsure, the person is told", and decision 5's amendment says the same, including tries that never reached Twilio; the query (`attempts > 0`, kinds added and moved, bound parameters, `sql.identifier(jobSchema)`) is unchanged and runs only when the first two "believes" checks fail (send-worker-text.ts:103-107 short-circuits). Nothing new from the rewording; the case where the job is gone before the taken-off reads it is F-249.

8c/F-249 [P2] closed - A "new booking" or "moved" retry that runs after the booking left the person, but before the taken-off job, completes without asking Twilio, so the in-doubt job is gone and the person who may have the text gets no "off your day"

File: backend/lib/text/send-worker-text.ts:86-87, 103-107, 133-139; backend/lib/jobs/has-worker-text-in-doubt.ts:1-5; blueprint/context/current-feature.md (decision 5, amendment) **Found:** 2026-10-07 by re-review of 8c.3's second fixes (scope: 286078b..21409d8; lenses: quality, security, performance, tests)

Why it matters: F-245's repair reads a waiting retry from the runner's table, so it works only while that job is still there. A retry of an added or moved text whose booking is no longer the person's stops at whyNotDue (line 86-87, "the booking was cancelled" or "another person's now") before the findSentText check (133-139), returns, and graphile-worker deletes the job. If that retry runs before the cancel's taken-off job, the taken-off finds no entry, no job in doubt, and logs "the person never knew of this booking". The order is the one F-247's paths used: a retry whose run_at (its failure plus e^attempts seconds) came before the cancel's job, picked after it, which the up-to-two-second poll gap or a runner restart gives, and the runner picks by run_at. Also reached when the taken-off's own first try fails and its retry falls behind. Probed with a temporary test in worker-text-job.test.ts (restored, same sha256): book with Marco, his "new booking" taken but answer lost; holdRetries; cancel; releaseRetries; workDueJobs. Marco's texts: only "Summit Painting: new booking Mon Oct 5, 9:00am. ..."; no worker entries; "worker_removed not sent, the person never knew of this booking" logged; no jobs left. The booking is cancelled and his last text says it is his: the empty-house case decision 5's amendment says a lost answer never leads to. Same trigger class and window as F-247, hence P2. **Suggested fix:** F-245's second suggestion: on attempt > 1, an added or moved text that whyNotDue rejects still runs the findSentText check (its words built from the job's own change where the booking's time may have moved since) and records the text with `text.sequence` when it went, before returning, so a taken-off that runs after it reads the entry. Or keep the in-doubt fact past the job (for example the retry, when not due, re-adds nothing but records that it may have gone). Add a test of the same name: a "new booking" whose answer was lost, then a cancel, with the retry released ahead of the cancel's job, texts the person "off your day".

Resolution: Fixed 2026-10-08 with Frank's yes (keeps his rule "when unsure, tell him", no new decision): a retry of a "new booking" or "moved" whose booking was cancelled or is another person's now still asks Twilio before it ends, and records the text with the change it was sent for when it went, so the "off your day" after it reads the entry (send-worker-text.ts, leftThePerson and foundEarlier); decision 5's amendment says so. Tests: "a lost new booking found on its retry after a cancel still texts the person off your day" (the finding's probe: one GET, Marco gets new booking then off your day, entries added then removed, no jobs left) and "a new booking its retry finds never went sends no off your day after a cancel"; proved: skipping the new check fails both (2 failed, 32 passed; file restored, same sha256). Known limit: the retry looks for the words the booking gives now, so if its time also moved between the lost try and the retry, and the move's own text never ran before the cancel, the text is not found and no "off your day" goes. Re-review of 8c.3's third fixes (2026-10-08): closed. The defect is gone: a retry (attempt > 1) of a "new booking" or "moved" whose booking was cancelled or is another person's now (leftThePerson, send-worker-text.ts:77-78) asks Twilio and, when the text went, records it with `text.sequence` before ending (send-worker-text.ts:123-143, through foundEarlier at 116-121), so the cancel's "off your day" reads the entry. Proved: changing line 127 to `if (false && attempt > 1 && ...)` fails both new tests (worker-text-job.test.ts:932 and 955; 2 failed, 32 passed; file restored, same sha256). No new wrong outcome in the orders probed with temporary tests (restored, same sha256): the "off your day" first and the retry after it (no third text, entries removed 1 then added 0, and a move back to Marco still sends "new

booking"); Twilio's list failing on the not-due retry (the job stays with attempts 2, so the in-doubt check sends "off your day", and the retry, released, records the text once with no further text); Marco's worker settings gone before the retry (no Twilio call, "the booking was cancelled" logged); a first try not due (no Twilio call at all). The extra loads and the one Twilio GET run only on a retry of a text whose booking left the person. But the known limit named above is real and is not in decision 5, whose amendment still says a lost answer never leaves them driving to an empty house: F-251.

8c/F-250 [P3] closed - The comment and the Data / contracts line for a text a retry finds say it "went at its first try" and told "only the change it was sent for"; it may have gone at any earlier try and described a later move

File: backend/lib/text/send-worker-text.ts:124-125, 136; blueprint/context/current-feature.md:299-301 (Data / contracts, Timeline) **Found:** 2026-10-07 by re-review of 8c.3's second fixes (scope: 286078b..21409d8; lenses: quality, security, performance, tests) **Why it matters:** findSentText looks for the same words since the change was saved (`since: text.changedAt`), so the text it finds may be from try 2 after a try 1 that never reached Twilio, not the first. And an added text renders the booking as it is when it runs (line 117, 122), so a try after a later time move told the person that move, past `text.sequence` . The recorded number is therefore the least the text told them, not what it described. That is the safe direction today (it can only let a later "new booking" or "moved" repeat, and no wrong last text came from it in any lane order traced, apart from F-249), but the comment gives a reason that is not true, and the spec records it as the contract, so the next rule built on `sequence` would trust it as exact. **Suggested fix:** Reword both to what holds: a text a retry finds is recorded with the change it was sent for, the least it can have told them (it went at some earlier try, about this change or a later one); a lower number at worst lets a later text repeat. **Resolution:** Fixed 2026-10-08 (wording only): the comment at send-worker-text.ts (record) and the Data / contracts Timeline line now say `sequence` is the least the text told them: sent now, the booking as it is; found on a retry, the change it was sent for, since it went at some earlier try and may describe a later move, so a lower number at worst lets a later text repeat. Re-review of 8c.3's third fixes (2026-10-08): closed. Both now say what holds. The comment beside `record` (send-worker-text.ts:107-108) and the Timeline line (current-feature.md:299-305) call `sequence` the least the text told them: the booking as it is for a text sent now; for one a retry finds, the change it was sent for. That is true: findSentText matches the same words since `text.changedAt` (send-worker-text.ts:118), so a found text went at or after this change and described the booking at a move number at or past `text.sequence` , and every retry that finds one, due or not, records `text.sequence` (line 119). No em dashes in the range.

8c/F-251 [P3] accepted - A "new booking" whose answer was lost, then a time move and a cancel before its retry, is not found by the retry, which looks for the words the booking gives now, so the person gets no "off your day"; decision 5 still says a lost answer never leaves them driving to an empty house

File: backend/lib/text/send-worker-text.ts:87, 123-143; blueprint/context/current-feature.md:14 (Goal), 121-130 (decision 5, amendment) **Found:** 2026-10-08 by re-review of 8c.3's third fixes (scope: 5bfd94b..7ed3ef3; lenses: quality, security, performance, tests) **Why it matters:** F-249's fix renders the retry's words from the booking as it is now (workerMessage, send-worker-text.ts:87, `context.startsAt`), but the lost try rendered the time the booking had then. If the booking's time moved after the lost try and the move's own text never ran while the booking was still the person's, the retry looks for words that were never sent, finds nothing, ends, and leaves no job for `hasWorkerTextInDoubt`. Probed with a temporary test in `worker-text-job.test.ts` (restored, same sha256), F-249's own order plus one move: `loseFirst("Summit Painting: new booking")`; `book` with Marco; `workDueJobs`; `holdRetries`; `move` to 10:00 keeping Marco; `cancel`; `releaseRetries`; `workDueJobs`. One GET; Marco's texts: only "Summit Painting: new booking Mon Oct 5, 9:00am. ..."; no worker entries; no jobs left; logged "worker_added not sent, the booking was cancelled", "worker_moved not sent, the booking was cancelled" and "worker_removed not sent, the person never knew of this booking". The booking is cancelled and his last text sends him to Jane's at nine: the case the Goal (line 14) and decision 5's amendment ("a lost answer never leaves them driving to an empty house", and "records the text if it went") say cannot happen. F-249's Resolution names this limit, but only here, which Frank does not read; the spec states the opposite as the contract. It needs F-249's window plus a time move inside it, narrower than F-249, and the fixer disclosed it, hence P3 rather than P2. **Suggested fix:** Frank decides in decision 5: either accept it and say so in the amendment's "Accepted as is" (a retry that finds the booking no longer theirs looks for the words the booking gives now, so a lost "new booking" or "moved" followed by a time move and a cancel before its retry gets no "off your day"); or close it by keeping the doubt when the retry cannot know the words, for example when it is not due, finds nothing, and the booking's time moved since this change (`context.sequence > text.sequence`), record that the text may have gone (a marker the "believes" check reads, as it reads the waiting job today), so the "off your day" goes, in the "when unsure, tell him" direction. Add the probe above as a test either way. **Resolution:** Accepted by Frank 2026-10-08 as a known limit: it needs a lost Twilio answer (about 1 in 10,000 texts) and a time move plus a cancel in the seconds before the retry (about 1 in 10,000 bookings), roughly 1 in 100 million worker texts. Decision 5's amendment now lists it under accepted, so it no longer claims a lost answer never leads to an empty house in this order. No code change.

Independent review

Status: passed **Target commit:** 2ad0463ad27ca0a8ef73c5397ea3ace62a010489 **Base commit:** ddca0e18dc1ee067f8530a41fdedfdec7f94821c **Base ref:** main **Spec hash:** a990f513c4b74db71e9efa37b7c9e85eb9f1c481e8a7a6f66c6f6bae01fa284a **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** runtime default (exact model not known until reviewer starts) **Requested execution:** automatic **Requested at:** 2026-10-08T10:40:21.000Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-10-08T10:50:00.000Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Handoff

Review the active spec and the complete

`ddca0e18dc1ee067f8530a41fdedfdec7f94821c..2ad0463ad27ca0a8ef73c5397ea3ace62a010489` delta in a fresh session or isolated subagent without the builder conversation. Run all Audit lenses from scratch. Run Check when required above. Do not edit product code, accept findings, or reuse the existing findings as the review scope.

The untracked `blueprint/ai-voice-proposal.md` is the owner's own file, never committed and outside this work item; it is not part of the target and does not make the review stale.

Commands

- `npm run test --workspace=backend` (run 1): pass, 72 files, 791 tests
- `npm run test --workspace=backend` (run 2): pass, 72 files, 791 tests
- `npm run test --workspace=backend` (run 3): pass, 72 files, 791 tests
- `npm run test --workspace=@scheduleads-app/shared` : pass, 20 files, 154 tests
- `npm run build --workspace=backend` : pass
- `npm run build --workspace=frontend` : pass
- `npm run format:check` : pass

Evidence

- Freshness: `HEAD` = `2ad0463ad27ca0a8ef73c5397ea3ace62a010489`; `git merge-base main HEAD` = `ddca0e18dc1ee067f8530a41fdedfdec7f94821c`; sha256 of current-feature.md = `a990f513c4b74db71e9efa37b7c9e85eb9f1c481e8a7a6f66c6f6bae01fa284a`; working tree differs only in review.md and the owner's untracked ai-voice-proposal.md named in the Handoff.
- Delta reviewed in full (35 files, 18 commits): the worker_text_settings table, migration 0021 (snapshot prevId chains to 0020), validation schema and its test, the seed, the shared phone pattern and textable-number schema extracted from 8b; findWorkerTextContext, findWorkerTextSettings, findWorkerTextsTold, renderWorkerText, joinCutWords, sendWorkerText; the worker_text job, its lane, hasWorkerTextInDoubt, enqueueWorkerText; book-time, move-booking and cancel-booking enqueueing in their own transactions; all three new test files; build-plan item 12.
- Security: every read is scoped by organizationId (context, settings, timeline); the job payload carries ids and times only; the activity payload and log reasons carry no phone, name, address or words; typed fields go through plainText; the customer's phone and booking link never enter the text (asserted in the form test); a worker phone that is any business's texting number is refused by findTextingBusiness before sending; the database check and composite foreign key refuse a malformed phone and another business's person (tests).

- Logic traced by hand against decision 5 across lane orders: away and back, quick double move, cancel after move, cancel before added, lost answer then cancel, lost answer then move and back; all match the spec, with F-251 the accepted gap and F-252 the one new gap found.
- Tests: no `.only` , `.skip` or `.todo` ; each Done when bullet of 8c.1 to 8c.3 has a named test; backend jobs run in a per-worker schema (vitest.setup.ts), so the file's `delete from _private_jobs` does not touch other files; three consecutive full backend runs were green.
- Performance: a worker text makes a handful of indexed single-row reads plus one jobs-table scan only for "off your day" (short-circuited after the timeline check); no unbounded loop beyond the word-cutting loops, which shrink by one word per pass.

Findings

- F-252 [P3] open (new): a "new booking" or "moved" still in doubt keeps counting after the person was told the booking is off, so back and away again before its texts run sends a second "off your day".
- No P0 or P1 open or fixed. No earlier entry changed status in this pass.

Remaining risk

- Spec 8c.1's "db:migrate and db:seed on a fresh scheduleads_dev, the second seed run changes nothing" was not rerun here: rebuilding the dev database was outside this review's limits. The seed's insert uses `onConflictDoNothing` on `personId`, which reads correctly.
- No real Twilio send was exercised (every test fakes it, as the spec requires); `findSentText`'s matching against Twilio's real list is covered only by the faked answers.
- The dashboard activity record (`blueprint/.state/run.json`) was not written by the reviewer, since this review was limited to `findings.md` and `review.md`.